

Compreendendo as *roles*

Toda chamada ao modelo é uma **lista de mensagens**, e cada mensagem tem um papel (*role*): `system` dá as regras, `user` faz o pedido e `assistant` guarda o que o modelo já respondeu. Nesta aula, com o notebook `01-roles.ipynb`: a mesma pergunta com e sem `system`, um histórico de conversa montado à mão e um `system` prompt refinado.

messages

system

user

assistant

Histórico

System prompt refinado

Formação Claude Code 2026 · Baseado em console.groq.com/docs (Text Generation, API Reference) consultado em outubro de 2026 · Código e saídas reais gravadas no notebook `01-roles.ipynb` (modelo `openai/gpt-oss-120b`)

A primeira célula de *todo notebook*

01-roles.ipynb · célula 1

```
from dotenv import load_dotenv
from groq import Groq
import os
load_dotenv()

client = Groq(
    api_key=os.environ.get("GROQ_API"), # This is the default and
    can be omitted
)
```

O comentário em inglês diz que o parâmetro pode ser omitido, mas isso só vale para a variável padrão `GROQ_API_KEY`. Como o curso usa `GROQ_API`, mantenha o `api_key=`.

- ✓ `load_dotenv()` lê o arquivo `.env` da pasta e coloca a chave no ambiente (aula 02)
- ✓ `os.environ.get("GROQ_API")` busca a chave sem escrevê-la no código
- ✓ `client` é o objeto que conversa com a API do Groq; ele é criado uma vez e reaproveitado
- ✓ Rode esta célula **antes** das outras: as demais usam `client`

Todos os notebooks do módulo começam com esta mesma célula.

Uma conversa é uma *lista de mensagens*

SYSTEM

Regras de comportamento: persona, tom, formato, limites. Vale para a conversa inteira.

USER

O pedido de quem usa: a pergunta, os dados, a tarefa.

ASSISTANT

Respostas do modelo. Você as envia de volta para o modelo lembrar o que já disse.

Formato de cada mensagem

```
messages=[
  {"role": "system", "content": "regras..."},
  {"role": "user", "content": "pergunta..."},
]
```

- ✓ A ordem importa: o modelo lê a lista de cima para baixo
- ✓ O `system` costuma vir primeiro e uma vez só
- ✓ Depois, `user` e `assistant` se alternam

Pedido simples, *sem regras*

célula 4

```
chat_completion = client.chat.completions.create(
    messages=[
        {
            "role": "user",
            "content": "Me explique de forma reduzida o que são as
LLMs",
        }
    ],
    model="openai/gpt-oss-120b",
)
print(chat_completion.choices[0].message.content)
```

- ✓ Uma única mensagem `user`
- ✓ `model` escolhe o modelo hospedado no Groq

SAÍDA GRAVADA NO NOTEBOOK

As **LLMs (Large Language Models)** são modelos de inteligência artificial treinados com enormes quantidades de texto para aprender padrões da linguagem humana. Eles usam arquiteturas de redes neurais, geralmente do tipo **Transformer**, que permitem processar e gerar texto de forma coerente e contextualizada.

- **Escala**: possuem bilhões a trilhões de parâmetros, o que aumenta sua capacidade de capturar nuances linguísticas.
- **Treinamento**: aprendem de forma não supervisionada, prevendo a próxima palavra em frases de vastos corpora (livros, artigos, sites, etc.).
- **Uso**: são a base de assistentes de texto, chatbots, tradutores automáticos, resumidores, geradores de código, entre outras aplicações.

Em resumo, LLMs são “cérebro” de IA que, a partir de muito texto, conseguem entender e produzir linguagem natural de forma impressionante.

A mesma pergunta, *com um papel definido*

célula 6

```
chat_completion = client.chat.completions.create(
    messages=[
        {
            "role": "system",
            "content": "Você é um especialista em IA que responde de
forma clara e objetiva."
        },
        {
            "role": "user",
            "content": "Me explique de forma reduzida o que são as
LLMs",
        }
    ],
    model="openai/gpt-oss-120b",
```

A pergunta do `user` é idêntica à da seção anterior. Só entrou a mensagem `system`.

SAÍDA GRAVADA NO NOTEBOOK

As **LLMs (Large Language Models)** são modelos de inteligência artificial baseados em redes neurais muito grandes, treinados com enormes volumes de texto (bilhões de palavras). Esse treinamento permite que eles:

1. **Aprendam padrões linguísticos** - gramática, vocabulário, estilo e relações semânticas.
2. **Entendam contexto** - analisam sequências de palavras para captar o sentido de frases e parágrafos.
3. **Gerem texto** - produzem respostas, resumos, traduções, códigos, entre outros, de forma coerente e fluente.

Em resumo, uma LLM é um modelo de IA que, a partir de vastos dados textuais, consegue compreender e gerar linguagem natural para diversas aplicações.

Respostas de LLM variam a cada execução: compare o estilo, não as palavras.

O *assistant* como memória da conversa

célula 8

```
chat_completion = client.chat.completions.create(
    messages=[
        {
            "role": "system",
            "content": "Você é um especialista em IA que responde de forma clara e objetiva."
        },
        {
            "role": "user",
            "content": "Explique a importância de LLMs com baixa latência."
        },
        {
            "role": "assistant",
            "content": "LLMs com baixa latência permitem respostas mais rápidas, o que melhora a experiência do usuário."
        },
        {
            "role": "user",
            "content": "Pode dar exemplos de aplicações práticas?"
        }
    ],
```

INÍCIO DA SAÍDA GRAVADA

****Exemplos de aplicações práticas que exigem LLMs de baixa latência****

Área	Aplicação	Por que a latência baixa é crítica
****Assistentes virtuais****	Chatbots de suporte ao cliente, assistentes de voz (Alexa, Google Assistant)	Respostas quase instantâneas mantêm a conversa fluida e evitam frustração do usuário.
...

- ✓ "Pode dar exemplos?" só faz sentido porque a pergunta anterior está na lista
- ✓ A resposta de **assistant** foi escrita à mão: o modelo a trata como algo que ele mesmo disse
- ✓ A API **não guarda** a conversa: um chatbot reenvia o histórico a cada chamada

Um system prompt *com instruções concretas*

célula 10

```
chat_completion = client.chat.completions.create(
    messages=[
        {
            "role": "system",
            "content": (
                "Você é um especialista em Inteligência Artificial focado em explicar conceitos "
                "de forma clara, estruturada e prática. Sempre inicie suas respostas com um "
                "resumo em até 3 linhas, depois detalhe com exemplos reais e aplicações no mundo "
                "dos negócios e tecnologia. Evite termos excessivamente técnicos sem explicação "
                "e, quando necessário, use analogias simples. "
                "Se o usuário pedir código, forneça exemplos em Python bem comentados."
            ),
        },
        {
            "role": "user",
            "content": "Explique a importância de LLMs com baixa latência."
        }
    ],
    model="openai/gpt-oss-120b",
```

INÍCIO DA SAÍDA GRAVADA

```
**Resumo (até 3 linhas)**
LLMs (Large Language Models) com baixa latência respondem rapidamente, o que melhora a experiência do usuário, permite decisões em tempo real e reduz custos operacionais. Em ambientes críticos, como atendimento ao cliente ou sistemas de controle, a rapidez é tão importante quanto a precisão.
...
```

- ✓ Pediu "resumo em até 3 linhas" primeiro: a resposta abre com **Resumo (até 3 linhas)**
- ✓ Pediu exemplos reais e de negócios: vieram tabela e casos de uso
- ✓ Os parênteses juntam várias strings numa só, para o texto caber na tela

Quanto mais concreta a regra, mais fácil conferir se ela foi seguida.

De onde sai o *texto da resposta*

O caminho até o texto

```
chat_completion.choices[0].message.content
```

CHOICES
lista de respostas



[0]
a primeira (e única)



MESSAGE.CONTENT
o texto

- ✓ `choices` é uma lista porque o formato da API permite pedir várias respostas
- ✓ No Groq, só `n=1` é aceito: sempre há uma resposta, a de índice 0
- ✓ `message.role` vem como `"assistant"`: é essa mensagem que você guardaria no histórico
- ✓ O texto chega em Markdown (`**negrito**`, tabelas com `|`); o `print` mostra as marcas cruas

Monte o seu *tutor*

Exercício: numa célula nova, crie um tutor de Python para iniciantes. O `system` deve exigir respostas curtas, um exemplo de código e uma pergunta no final para checar o entendimento.

Ponto de partida

```
resp = client.chat.completions.create(
    model="openai/gpt-oss-120b",
    messages=[
        {"role": "system", "content": "..."},
        {"role": "user", "content": "O que é uma lista?"},
    ],
)
print(resp.choices[0].message.content)
```

- ✓ Rode duas vezes e veja se as três regras foram cumpridas
- ✓ Depois acrescente a resposta como `assistant` e uma nova pergunta do `user` ("E uma tupla?")
- ✓ Teste tirar o `system`: o que muda?

Cada execução consome cota do plano gratuito (aula 01). Prefira poucas execuções bem pensadas.

Teste o que *ficou*

1. Onde vão as regras de comportamento do modelo?

A) user

B) system

C) assistant

D) model

B.

2. Para o modelo lembrar a conversa, você:

A) Não faz nada

B) Reenvia o histórico em messages

C) Aumenta a temperatura

D) Cria outro client

B. A API não guarda estado entre chamadas.

3. Uma mensagem com role "assistant" representa:

A) O usuário

B) Uma regra

C) Uma resposta anterior do modelo

D) Um erro

C.

4. O texto gerado fica em:

A) chat_completion.text

B) choices[0].message.content

C) messages[0]

D) client.content

B.

Quem fala o quê *na conversa*

O `system` define as regras, o `user` faz o pedido e o `assistant` guarda as respostas anteriores. A API não tem memória: o histórico é a própria lista `messages`. Um `system` com instruções concretas muda a forma da resposta de um jeito que dá para conferir.

`system`

`user`

`assistant`

Histórico reenviado

`choices[0].message.content`

Próxima aula → 05 · Entendendo os Parâmetros

Fontes: console.groq.com/docs/text-chat; console.groq.com/docs/api-reference (chat completions: messages, n); console.groq.com/docs/openai, consultados em outubro de 2026. Código e saídas: notebook `01-roles.ipynb` desta aula.